Host Header Injection Testing

Using BurpSuite Community Edition

Conducted on PortSwigger Web Security Academy

Prepared by: MOHAMMED ALI

Institution: BOSTON INSTITUTE OF ANALYTICS

Assessment Type	Web Application Penetration Testing — Host Header Injection
Scope	Password reset flow, authentication endpoints, HTTP header handling
Methodology	OWASP Testing Guide (OTG), PortSwigger Research, Manual Exploitation
Tools Used	BurpSuite Community Edition — Proxy, Repeater, HTTP History
Testing Environment	PortSwigger Web Security Academy (Controlled Lab Environment)
Date	May 2026
Report Classification	CONFIDENTIAL — Restricted Distribution
Severity	HIGH — Immediate Remediation Required

CONFIDENTIAL — PENETRATION TESTING ASSESSMENT REPORT

Abstract

Host Header Injection is a class of web application vulnerability that arises when a server-side application uses the HTTP Host header to construct URLs, redirect responses, or make security decisions without adequately validating its value. This project documents a structured penetration testing exercise conducted against intentionally vulnerable web applications in the PortSwigger Web Security Academy environment, specifically targeting Host Header Injection vectors.

Testing was performed using BurpSuite Community Edition, leveraging the Proxy and Repeater modules to intercept live HTTP traffic, modify Host-related headers, and observe application responses. The primary attack vector investigated was password reset poisoning, whereby a manipulated Host header causes the application to embed an attacker-controlled domain within a generated password reset link.

Testing demonstrated that the target application accepted arbitrary Host header values, processed X-Forwarded-Host headers without strict validation, and exhibited inconsistent behavior when presented with duplicate Host headers. These findings indicate a High severity vulnerability with meaningful potential for credential theft, phishing, and authentication flow compromise.

This report covers the full assessment lifecycle including environment setup, testing methodology, payload documentation, screenshot evidence, risk analysis, and remediation recommendations.

Table of Contents

1. Introduction

The HTTP Host header is a mandatory component of every HTTP/1.1 request. It specifies the domain that the client intends to communicate with, and is relied upon by web servers, load balancers, and reverse proxies to route incoming traffic to the correct backend application. When web applications trust this header without appropriate validation, attackers may manipulate its value to influence server-side behavior in unintended ways.

Host Header Injection represents a family of vulnerabilities that exploit this trust. In practice, the most impactful manifestations include password reset poisoning, web cache poisoning, server-side request forgery (SSRF) via backend service routing, and open redirect via reverse proxy confusion. These vulnerabilities can lead to account takeover, credential theft, and broad compromise of user sessions.

This project was conducted as a structured, documented penetration testing exercise using BurpSuite Community Edition against PortSwigger Web Security Academy lab environments. The labs simulate real-world vulnerable application behavior, allowing safe and legal hands-on testing. The exercise covered basic Host header manipulation through to advanced forwarded header abuse and duplicate header ambiguity testing.

1.1 Background

Modern web infrastructure frequently involves multiple layers: a public-facing reverse proxy or load balancer that routes traffic, an application server that processes business logic, and various backend services. The Host header is often used across these layers to determine routing, construct absolute URLs in email links, or enforce access control decisions. When the innermost application layer blindly trusts the Host value without verifying it against a known-good list, the attack surface opens significantly.

Password reset poisoning, first documented widely by James Kettle of PortSwigger Research, is perhaps the most dangerous consequence. The attacker triggers a password reset for a victim account while intercepting the request and replacing the legitimate domain in the Host header with an attacker-controlled domain. If the application constructs the reset link using the Host header value, the victim receives an email with a malicious reset URL that, when clicked, delivers the reset token directly to the attacker.

1.2 Project Scope

This assessment was limited entirely to intentionally vulnerable lab environments provided by PortSwigger Web Security Academy. No production systems, live users, or real organizational assets were targeted. All testing was conducted from a single analyst workstation using BurpSuite Community Edition and the PortSwigger-provided exploit server infrastructure.

2. Objectives

The objectives of this penetration testing project were:

- Configure BurpSuite Community Edition as a man-in-the-middle proxy and verify successful HTTPS traffic interception.
- Identify and analyze the Host header in captured HTTP requests directed at the target application.
- Test the application's response to arbitrary Host header values, including attacker-controlled domain names.
- Evaluate the application's handling of the X-Forwarded-Host header as an alternate injection vector.
- Assess the behavior of the target when presented with duplicate Host headers in a single HTTP request.
- Simulate password reset poisoning using PortSwigger's exploit server as the attacker-controlled domain.
- Document all testing steps, payloads, and observed responses with supporting screenshot evidence.
- Analyze identified vulnerabilities, assess their severity and exploitability, and recommend practical mitigations.

3. Scope of Testing

3.1 In-Scope

- PortSwigger Web Security Academy Host Header Injection lab environments
- Password reset functionality (/forgot-password endpoint)
- HTTP Host header manipulation
- X-Forwarded-Host header injection testing
- Duplicate Host header ambiguity testing
- Exploit server domain injection simulation

3.2 Out of Scope

- Any live production web application
- Real user accounts or organizational data
- Network infrastructure beyond the lab environment
- Any system not explicitly provided as part of the PortSwigger lab setup

4. Tools Used

The following tools and platforms were used throughout this assessment:

Tool / Resource	Purpose
BurpSuite Community Edition	Primary proxy and interception tool for capturing and analyzing HTTP/S traffic
BurpSuite Proxy Module	Intercept and inspect HTTP requests and responses in real time
BurpSuite Repeater Module	Manually modify and replay HTTP requests for header manipulation testing
BurpSuite HTTP History	Review historically captured requests and responses during testing sessions
PortSwigger Web Security Academy	Controlled, intentionally vulnerable lab environments for realistic testing
Exploit Server (PortSwigger)	Attacker-controlled domain server used to simulate Host header poisoning attacks
Burp Chromium Browser	Pre-configured browser for seamless BurpSuite proxy integration

5. Testing Methodology

Testing was conducted following a structured, phased methodology consistent with standard web application penetration testing practices.

5.1 Phase 1 — Environment Configuration

BurpSuite Community Edition was installed and configured on the analyst workstation. The embedded Burp Chromium Browser was used to route all web traffic through the Burp proxy listener, configured on 127.0.0.1:8080. HTTPS interception was enabled, allowing BurpSuite to decrypt, inspect, and modify TLS-encrypted traffic. The PortSwigger Web Security Academy Host Header Injection labs were accessed and confirmed as active.

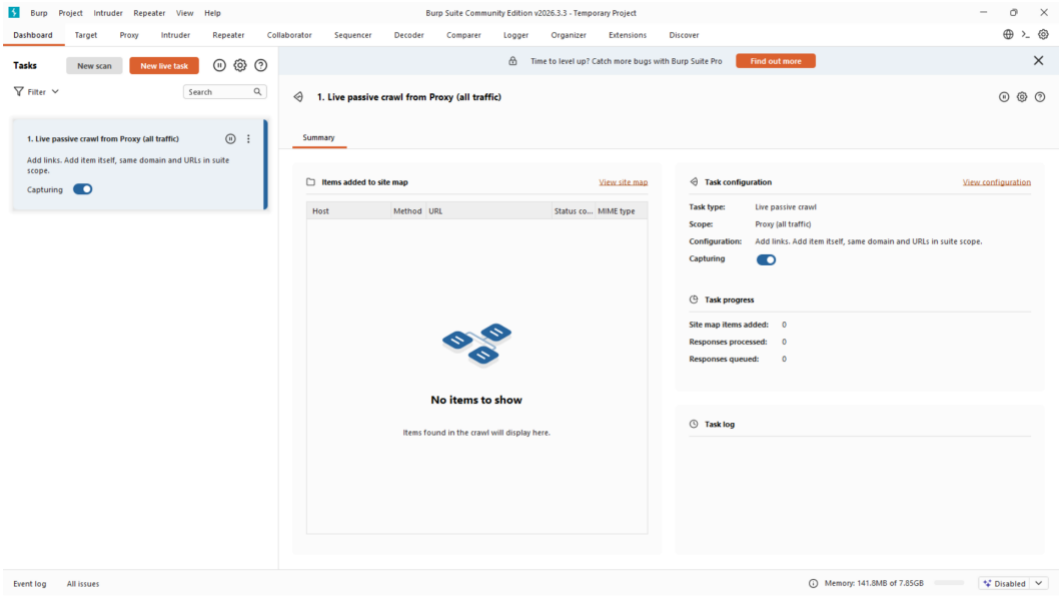

Figure 1: BurpSuite Community Edition — Dashboard showing proxy running

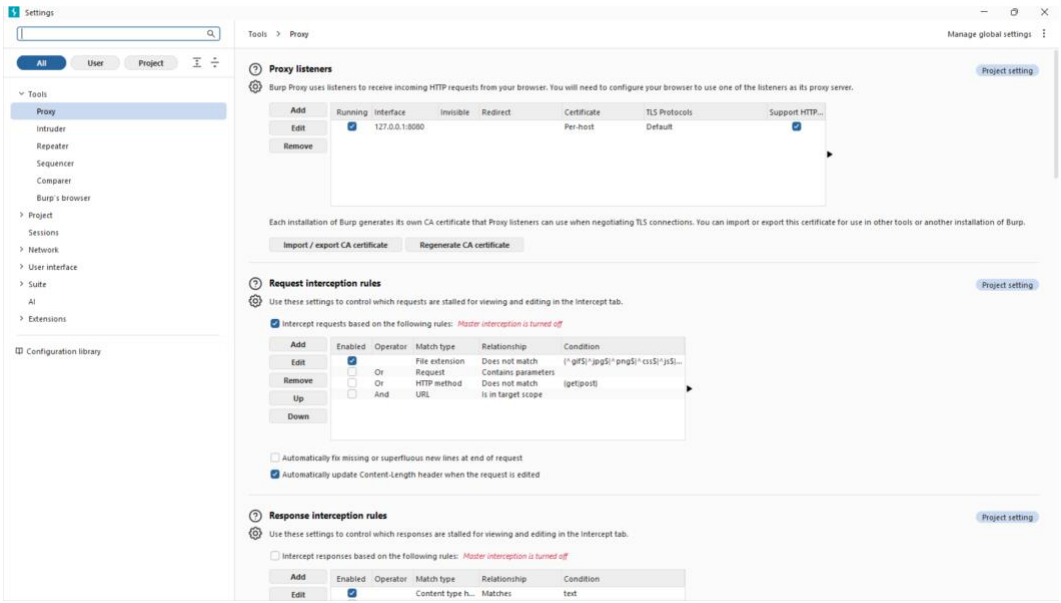

Figure 2: BurpSuite Proxy Listener configured on 127.0.0.1:8080

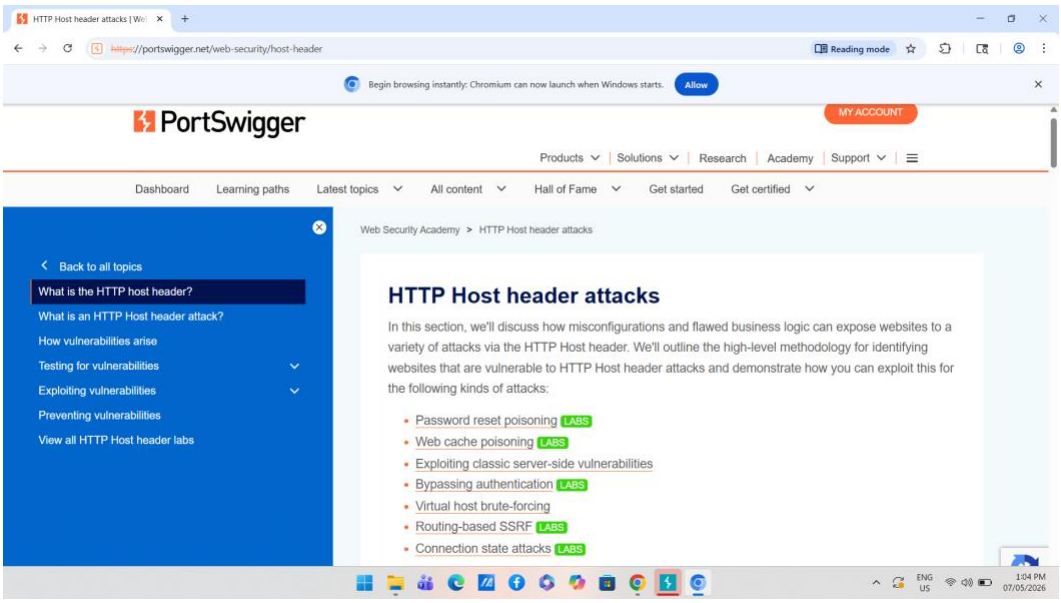

Figure 3: PortSwigger Web Security Academy — Host Header Injection Labs

5.2 Phase 2 — Traffic Interception and Request Analysis

With the proxy operational, the analyst navigated to the target lab application and triggered a password reset request by submitting the "Forgot password" form. The outgoing HTTP POST request to /forgot-password was intercepted in real time via BurpSuite Proxy. The Host header value was identified and recorded as the baseline for subsequent manipulation tests.

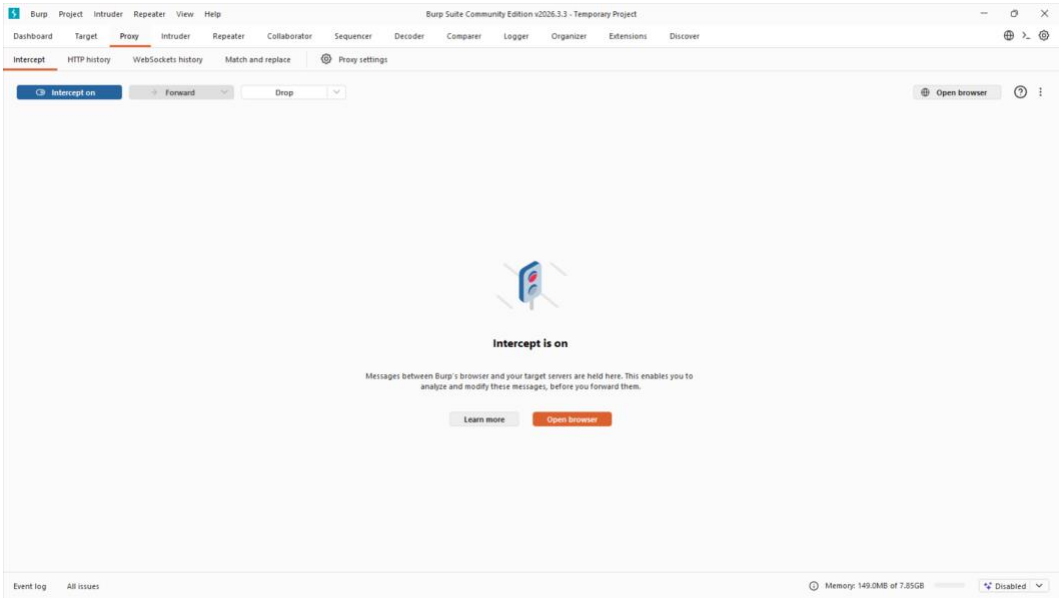

Figure 4: Intercept enabled — live HTTP traffic captured in BurpSuite Proxy

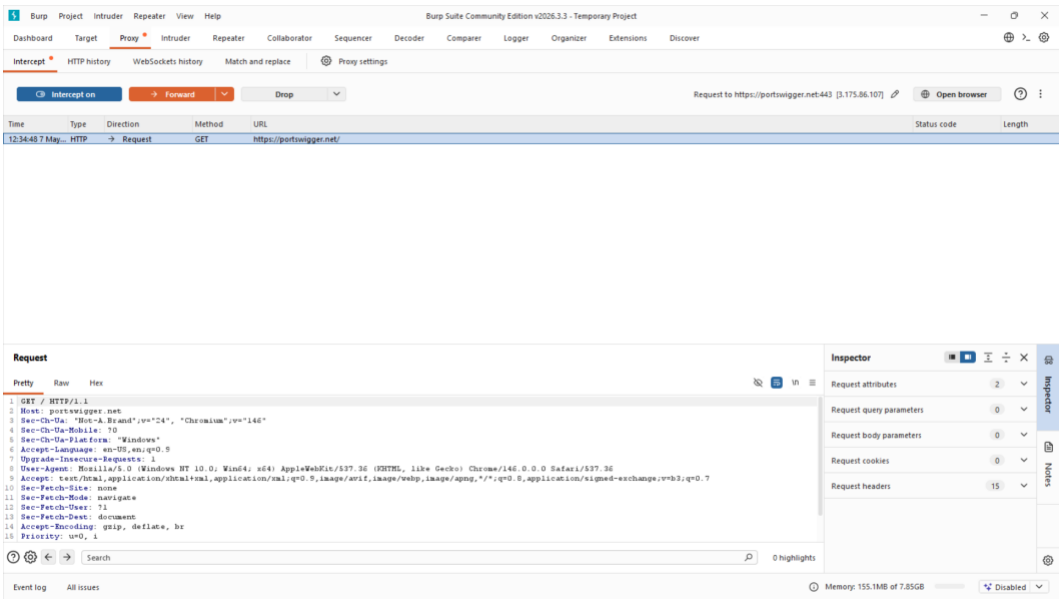

Figure 5: Intercepted HTTP request showing the Host header value

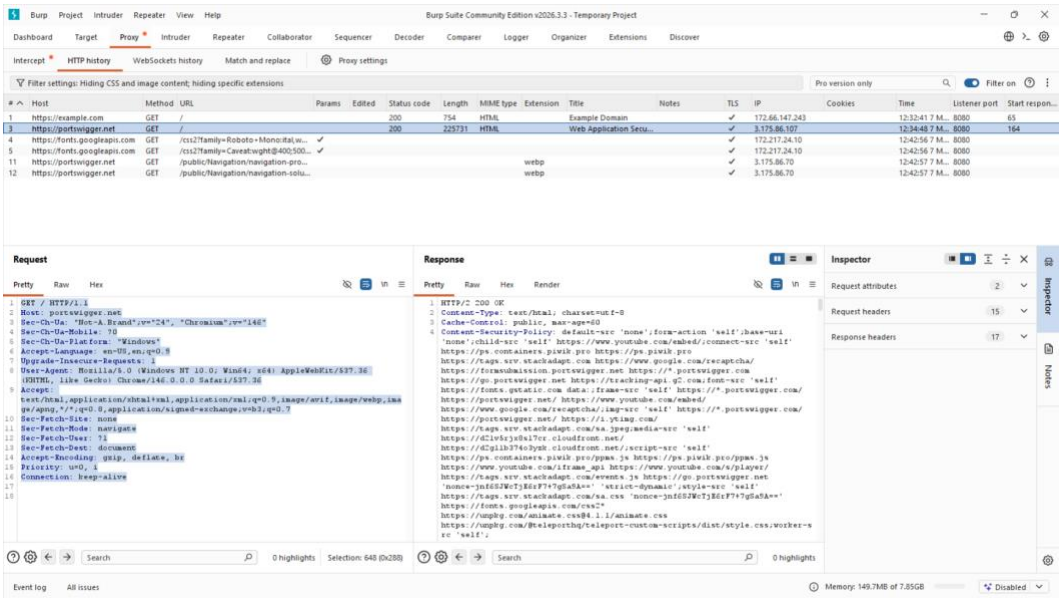

Figure 6: Request analysis — reviewing headers, parameters, and Host value

5.3 Phase 3 — Basic Host Header Manipulation

The intercepted password reset request was forwarded to BurpSuite Repeater. Within Repeater, the Host header value was manually modified to an attacker-controlled domain (attacker.com). The modified request was sent to the server and the response was recorded. The process was repeated using the PortSwigger exploit server domain as the injected Host value.

Figure 7: Lab environment ready — target application loaded for testing

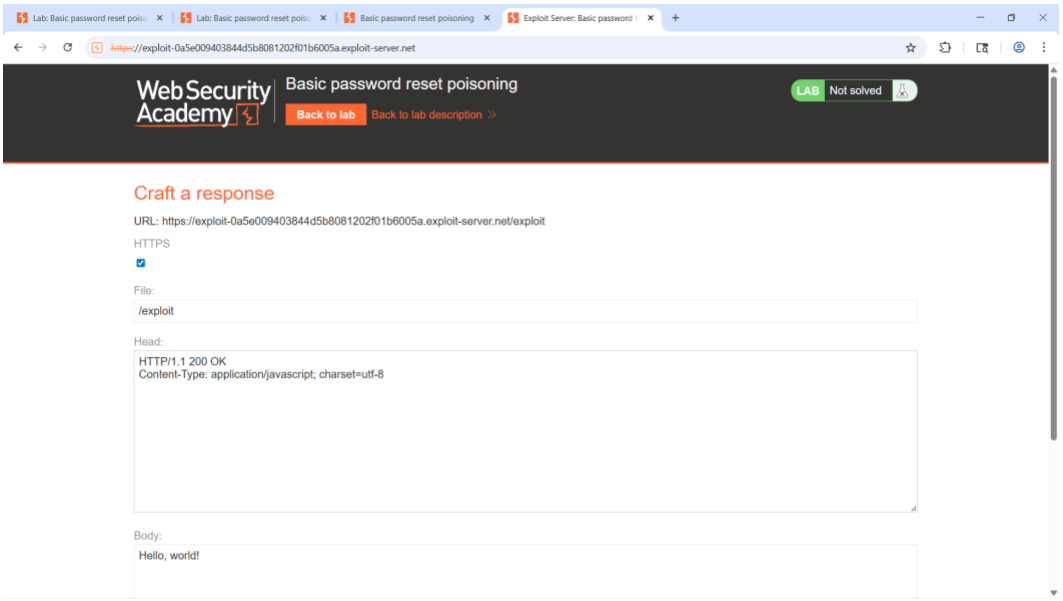

Figure 8: PortSwigger Exploit Server — attacker-controlled domain configured

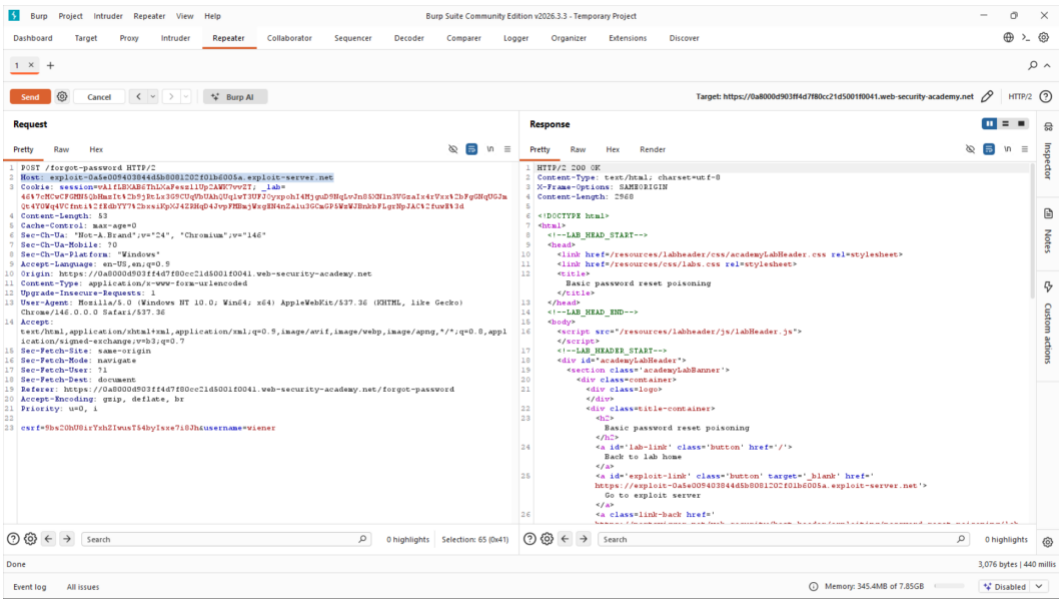

Figure 9: Host Header Injection payload sent via BurpSuite Repeater

5.4 Phase 4 — X-Forwarded-Host Header Testing

An additional X-Forwarded-Host header was injected into subsequent requests to test whether the application trusted forwarded header values from intermediate proxies. Two payloads were evaluated: a generic attacker-controlled domain and the PortSwigger exploit server domain. Both requests returned HTTP/2 200 OK responses, indicating that the application processed X-Forwarded-Host values without restriction.

5.5 Phase 5 — Duplicate Host Header Testing

A duplicate Host header scenario was tested by sending a request that contained two Host headers simultaneously: the legitimate lab domain and an attacker-controlled value. This scenario exploits parsing inconsistencies between different layers of the network stack, where the reverse proxy and the backend application may each honor a different Host header value. The application exhibited abnormal or blank response behavior, indicating parsing ambiguity.

5.6 Phase 6 — Final Validation Testing

A final round of testing compared application behavior using manipulated Host headers against the baseline valid Host header. Manipulated requests either returned inconsistent results or caused the server to hang. The baseline valid Host header produced consistent HTTP/2 200 OK responses, confirming that the anomalies observed were directly attributable to the injection payloads.

6. Vulnerability Overview

6.1 What is Host Header Injection?

Host Header Injection occurs when a web application incorporates the value of the HTTP Host header into server-side logic — such as URL generation, redirect construction, or access control decisions — without first validating it against a trusted list of acceptable domains. Since the HTTP Host header is entirely user-controlled, any application functionality that trusts it implicitly is exposed to manipulation.

The vulnerability is particularly dangerous when it affects security-sensitive operations like password reset link generation, account confirmation emails, or OAuth redirect URIs. In these cases, an attacker who can poison the Host header can steer the application into sending sensitive data to an attacker-controlled server.

6.2 Attack Vectors Demonstrated

Direct Host Header Manipulation:

Replacing the Host header value with an attacker-controlled domain. If the application uses this header to generate a password reset URL, the victim's reset token is sent to the attacker's server when the victim clicks the link.

X-Forwarded-Host Injection:

Adding an X-Forwarded-Host header alongside a legitimate Host header. Many backend applications trust this header as a signal of the original client-facing host when sitting behind a reverse proxy, making it an effective secondary injection point.

Duplicate Host Header Injection:

Sending two Host headers within a single request exploits RFC parsing ambiguity between front-end and back-end systems. The reverse proxy may process one Host value while the backend processes another, enabling routing confusion and cache poisoning.

7. Practical Testing Steps

1. Launched BurpSuite Community Edition and confirmed the proxy listener was active on 127.0.0.1:8080.
2. Opened PortSwigger Web Security Academy in the Burp Chromium Browser and navigated to the Host Header Injection lab.
3. Enabled intercept in BurpSuite Proxy and submitted the "Forgot password" form to capture the outgoing request.
4. Inspected the intercepted request in BurpSuite Proxy and identified the Host header value.
5. Forwarded the request to BurpSuite Repeater for controlled, repeatable manipulation.
6. Modified the Host header to "attacker.com" and sent the request. Observed and recorded the server response.
7. Modified the Host header to the PortSwigger exploit server domain and sent the request. Observed and recorded the server response.
8. Added an X-Forwarded-Host header with "attacker.com" while preserving a valid Host header. Sent and recorded response.

9. Added an X-Forwarded-Host header with the exploit server domain. Sent and recorded response.
10. Constructed a request containing two Host headers (legitimate domain + attacker.com). Sent and recorded the response for parsing inconsistency analysis.
11. Performed final validation: sent a request with manipulated Host and compared against a request with the valid Host header.
12. Collected and organized all screenshots, payloads, and observations for report preparation.

8. Payloads Used

8.1 Basic Host Header Manipulation

Payload 1 — Generic Attacker Domain:

```
POST /forgot-password HTTP/2
Host: attacker.com
Content-Type: application/x-www-form-urlencoded

username=victim@mail.com
```

Purpose: Test whether the application accepts an arbitrary Host header value and uses it in password reset link construction.

Observed Behavior: Application processed the password reset request successfully. HTTP/2 200 OK returned.

Payload 2 — Exploit Server Domain:

```
POST /forgot-password HTTP/2
Host: exploit-0a5e009403844d5b8081202f01b6005a.exploit-server.net
Content-Type: application/x-www-form-urlencoded

username=victim@mail.com
```

Purpose: Simulate realistic password reset poisoning using an attacker-controlled exploit server domain.

Observed Behavior: Server accepted and processed the request with the manipulated Host value.

8.2 X-Forwarded-Host Injection

Payload 3 — Forwarded Header with Generic Domain:

```
POST /forgot-password HTTP/2
Host: 0ac7009903dcb58e8284a1b6007700b8.web-security-academy.net
X-Forwarded-Host: attacker.com
Content-Type: application/x-www-form-urlencoded
```

Observed Behavior: Application accepted the additional X-Forwarded-Host header. HTTP/2 200 OK returned.

Payload 4 — Forwarded Header with Exploit Server Domain:

```
POST /forgot-password HTTP/2
Host: 0ac7009903dcb58e8284a1b6007700b8.web-security-academy.net
X-Forwarded-Host: exploit-0a22008503e4b5b882d2a03601bc0079.exploit-server.net
```

Observed Behavior: Application processed the request using the attacker-controlled forwarded host value. HTTP/2 200 OK returned.

8.3 Duplicate Host Header

Payload 5 — Duplicate Host Headers:

```
POST /forgot-password HTTP/2
Host: 0ac7009903dcb58e8284a1b6007700b8.web-security-academy.net
Host: attacker.com
Content-Type: application/x-www-form-urlencoded
```

Observed Behavior: Application/server response became inconsistent or blank. This indicates possible parsing ambiguity between front-end and back-end layers.

8.4 Baseline Validation

Baseline Payload — Valid Host Header:

```
POST /forgot-password HTTP/2
Host: 0ad100fc0490777a82fc5b7600a30096.web-security-academy.net
Content-Type: application/x-www-form-urlencoded
```

Observed Behavior: Normal password reset functionality. HTTP/2 200 OK with expected response body.

9. Observations and Results

The following observations were recorded across the testing phases:

Observation 1: Manipulated Host Headers Accepted

When the Host header in a password reset POST request was replaced with an arbitrary attacker-controlled domain, the application continued processing the request and returned HTTP/2 200 OK. This indicates that the server does not validate or whitelist accepted Host values before processing security-sensitive operations.

Observation 2: X-Forwarded-Host Not Strictly Validated

Adding an X-Forwarded-Host header with attacker-controlled values alongside a legitimate Host header resulted in HTTP/2 200 OK responses. The application trusted the forwarded header values without verification, extending the injection surface beyond the primary Host header.

Observation 3: Duplicate Host Headers Caused Parsing Inconsistency

Sending two Host headers in a single request caused abnormal or blank server responses. This behavior suggests that the reverse proxy and backend application process different Host values from the same request, creating a discrepancy that could be exploited for routing manipulation or cache poisoning.

Observation 4: Baseline Behavior Confirmed Normal

Requests sent with the legitimate lab domain in the Host header produced consistent, expected HTTP/2 200 OK responses. This confirms that the anomalies observed in previous tests are directly attributable to the injection payloads and not to general application instability.

Observation 5: Final Validation Confirmed Continued Vulnerability

Final validation testing reconfirmed that manipulated Host values caused the server to either wait indefinitely or produce inconsistent behavior, while the valid Host header continued to produce normal responses. This consistent differential reinforces the conclusion that Host header handling is insufficiently validated.

10. Risk Analysis

The vulnerabilities identified during this assessment carry significant security implications for any real-world application exhibiting similar behavior. The risk assessment below evaluates each attack scenario by impact, likelihood, and overall severity.

Risk	Impact	Likelihood	Severity
Password Reset Poisoning	Critical	High	High
Credential Theft	Critical	High	High
Phishing via Poisoned Links	High	High	High
Reverse Proxy Confusion	Medium	Medium	Medium
Cache Poisoning	High	Medium	High
Authentication Manipulation	Critical	Medium	High

10.1 Detailed Risk Descriptions

Password Reset Poisoning:

An attacker intercepts a password reset request and injects their controlled domain into the Host header. The application generates a reset link referencing the attacker's domain and sends it to the victim's email. When the victim clicks the link, the reset token is delivered to the attacker, enabling full account takeover.

Reverse Proxy Confusion:

Duplicate Host headers can cause front-end and back-end system components to interpret the request differently. This may result in unauthorized access to internal services, incorrect routing to unintended backend hosts, or bypassing IP-based access controls.

Web Cache Poisoning:

If the application caches responses that include content derived from the Host header, an attacker could poison the cache with malicious content. Subsequent legitimate users fetching the same cached resource would receive the attacker-injected content.

11. Mitigation Strategies

The following remediation measures are recommended to address the identified vulnerabilities and harden the application against Host Header Injection attacks:

11.1 Strict Host Header Whitelist Validation

The application must maintain a pre-configured whitelist of trusted domain names and validate every incoming Host header value against this list. Any request bearing a Host value not present in the whitelist should be rejected with a 400 Bad Request response. Never dynamically construct URLs, redirect targets, or email link content using the raw Host header value.

11.2 Reject or Strip Untrusted Forwarded Headers

X-Forwarded-Host, X-Forwarded-For, X-Host, and similar headers should not be trusted by the application server unless they originate from a known, controlled reverse proxy. Configure the reverse proxy to strip or overwrite these headers before forwarding requests to the backend, preventing external clients from injecting arbitrary values.

11.3 Deny Duplicate Host Headers

Web server configurations should explicitly reject HTTP requests containing more than one Host header. Apache, Nginx, and IIS all support configuration options to handle this scenario. Enforcing strict RFC 7230 compliance at the server layer eliminates the parsing ambiguity that enables duplicate-host attacks.

11.4 Use Absolute URLs From Configuration, Not Headers

For operations such as password reset, email confirmation, and OAuth redirect, the application should construct absolute URLs using a fixed, hard-coded base URL from the server-side configuration rather than deriving it from the incoming request's Host header. This eliminates the direct dependency on an attacker-controllable value.

11.5 Harden Reverse Proxy Configuration

Reverse proxies should be configured to normalize incoming headers, enforce canonical Host values before passing requests to the backend, and drop any request that does not match expected routing patterns. Regular security audits of reverse proxy and load balancer configurations are recommended.

11.6 Security Logging and Alerting

Implement logging for requests where the Host header does not match the expected canonical domain. Configure alerts for repeated manipulation attempts, which may indicate active exploitation attempts or reconnaissance activity targeting the application's Host handling logic.

11.7 Regular Penetration Testing

Host header handling should be explicitly included in periodic web application penetration testing engagements. Security regression tests validating that whitelisting, header stripping, and duplicate header rejection remain functional should be incorporated into the application's CI/CD pipeline where possible.

12. Final Findings Summary

Severity: HIGH — Immediate remediation recommended

- **The target application accepted arbitrary, attacker-controlled Host header values during password reset operations.**
- Password reset functionality continued processing normally after Host header modification, confirming exploitability of the password reset poisoning vector.
- X-Forwarded-Host headers were not strictly validated; attacker-controlled forwarded host values were accepted without restriction.
- Duplicate Host header submission caused abnormal or inconsistent server responses, indicating parsing ambiguity at the reverse proxy or application layer.
- Baseline testing with valid Host headers confirmed stable application behavior, isolating observed anomalies to the injection payloads specifically.
- The combination of these findings creates a realistic, exploitable attack chain targeting end-user credential security.

Evidence collected: BurpSuite Repeater request/response captures, HTTP response analysis, validation screenshots, and documented payload repository.

13. Conclusion

This project successfully demonstrated the identification and exploitation of Host Header Injection vulnerabilities using BurpSuite Community Edition against intentionally vulnerable lab environments. The hands-on testing exercise covered the complete vulnerability lifecycle from environment setup and traffic interception through payload delivery and evidence documentation.

The findings confirm that improper Host header validation is a genuinely impactful vulnerability class. When an application trusts user-supplied Host header values in security-sensitive operations such as password reset link generation, the consequences for end users can include full account takeover through credential theft, targeted phishing via poisoned reset emails, and broader infrastructure compromise through cache poisoning or reverse proxy confusion.

The X-Forwarded-Host and duplicate Host header tests further demonstrate that the injection surface extends beyond the primary Host header itself, requiring comprehensive server-side controls rather than point fixes. Organizations that deploy applications behind reverse proxies or CDNs should pay particular attention to header normalization and whitelist enforcement at every layer of their infrastructure.

From a skills and learning perspective, this project provided meaningful practical experience with BurpSuite's proxy and repeater workflows, HTTP request structure and header manipulation, web application vulnerability identification methodology, and professional penetration testing documentation. These competencies form a strong foundation for continued security research and professional security assessment work.

14.References

1. PortSwigger Web Security Academy. *HTTP Host Header Attacks*. https://portswigger.net/web-security/host-header
2. PortSwigger Web Security Academy. *Password Reset Poisoning via Host Header*. https://portswigger.net/web-security/host-header/exploiting/password-reset-poisoning
3. OWASP. *Web Security Testing Guide — Testing for Host Header Injection*. https://owasp.org/www-project-web-security-testing-guide/
4. PortSwigger Ltd. *BurpSuite Community Edition Documentation*. https://portswigger.net/burp/documentation
5. RFC 7230 — *Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing*. https://tools.ietf.org/html/rfc7230